

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЁТ ПО ЗАДАНИЮ №6

**«Сборка многомодульных программ.
Вычисление корней уравнений и определённых
интегралов.»**

Вариант 9 / 1 / 1

Выполнил:
студент 104 группы
Буклов М. А.

Преподаватель:
Смирнов М. Б.

Москва
2026

Содержание

Постановка задачи	2
Математическое обоснование	3
Анализ функций	3
Нахождение отрезков для поиска корней	3
Метод деления отрезка пополам	3
Метод хорд	3
Обоснование выбора ε_1	4
Формула для площади	4
Обоснование выбора ε_2	4
Графики функций	4
Результаты экспериментов	5
Структура программы и спецификация функций	6
Сборка программы (Make-файл)	8
Отладка программы, тестирование функций	9
Тестирование <code>root</code>	9
Тестирование <code>integral</code>	9
Программа на Си и на Ассемблере	10
Анализ допущенных ошибок	11
Список цитируемой литературы	12

Постановка задачи

Требуется написать программу, вычисляющую площадь плоской фигуры, ограниченной тремя кривыми (вариант 9):

$$f_1(x) = \frac{3}{(x-1)^2 + 1}, \quad f_2(x) = \sqrt{x+0,5}, \quad f_3(x) = e^{-x}.$$

Программа должна:

1. найти абсциссы трёх точек попарного пересечения кривых с точностью ε_1 . Согласно варианту реализованы *два* метода приближённого решения уравнения $f(x) = g(x)$: метод деления отрезка пополам и метод хорд. Выбор метода выполняется на этапе компиляции через макрос препроцессора `-DMETHOD_CHORDS` (по умолчанию — деление пополам);
2. вычислить площадь фигуры как алгебраическую сумму определённых интегралов методом прямоугольников (средних точек) с точностью ε_2 .

Отрезки для поиска корней определяются аналитически заранее (с анализом смены знака разности функций). Точности ε_1 и ε_2 подбираются так, чтобы итоговая площадь была вычислена с точностью $\varepsilon = 10^{-3}$.

Функции f_1 , f_2 , f_3 реализованы на языке ассемблера NASM32 (архитектура IA-32, соглашение cdecl). Остальной код написан на языке Си. Сборка выполняется через `make`.

Математическое обоснование

Анализ функций

Функция $f_1(x) = 3/((x-1)^2 + 1)$ является колоколообразной: её максимум $f_1(1) = 3$ достигается при $x = 1$, а при $x \rightarrow \pm\infty$ функция стремится к нулю. Функция $f_2(x) = \sqrt{x+0,5}$ определена при $x \geq -0,5$ и монотонно растёт. Функция $f_3(x) = e^{-x}$ монотонно убывает, принимая значение 1 при $x = 0$.

Нахождение отрезков для поиска корней

Нужно найти три точки пересечения: $f_1 \cap f_3$, $f_2 \cap f_3$, $f_1 \cap f_2$. Для каждой пары введём разность $\varphi(x) = f_i(x) - f_j(x)$ и найдём отрезок, на котором она меняет знак.

Пересечение f_1 и f_3 (точка А). Вычислим значения $f_1 - f_3$ в нескольких точках:

$$\begin{aligned} x = -0,5 : \quad f_1 &= 0,923, \quad f_3 = 1,649 \quad \Rightarrow \quad f_1 - f_3 = -0,726 < 0, \\ x = 0 : \quad f_1 &= 1,500, \quad f_3 = 1,000 \quad \Rightarrow \quad f_1 - f_3 = +0,500 > 0. \end{aligned}$$

Знак сменился на $[-0,5; 0]$, поэтому ищем корень на этом отрезке.

Пересечение f_2 и f_3 (точка В). Вычислим $f_2 - f_3$:

$$\begin{aligned} x = 0 : \quad f_2 &= 0,707, \quad f_3 = 1,000 \quad \Rightarrow \quad f_2 - f_3 = -0,293 < 0, \\ x = 1 : \quad f_2 &= 1,225, \quad f_3 = 0,368 \quad \Rightarrow \quad f_2 - f_3 = +0,857 > 0. \end{aligned}$$

Ищем корень на $[0; 1]$.

Пересечение f_1 и f_2 (точка С). Вычислим $f_1 - f_2$:

$$\begin{aligned} x = 1 : \quad f_1 &= 3,000, \quad f_2 = 1,225 \quad \Rightarrow \quad f_1 - f_2 = +1,775 > 0, \\ x = 3 : \quad f_1 &= 0,600, \quad f_2 = 1,871 \quad \Rightarrow \quad f_1 - f_2 = -1,271 < 0. \end{aligned}$$

Ищем корень на $[1; 3]$.

Метод деления отрезка пополам

Пусть $\varphi(x) = f(x) - g(x)$ непрерывна на $[a, b]$ и $\varphi(a)\varphi(b) < 0$. На каждом шаге вычисляется середина отрезка $c = (a+b)/2$ и значение $\varphi(c)$; далее один из концов отрезка заменяется на c так, чтобы знак функции на концах по-прежнему оставался разным. Итерации продолжаются, пока $b - a > \varepsilon_1$, после чего возвращается $(a+b)/2$.

Метод хорд

Метод хорд (секущих) приближает корень точкой пересечения хорды, соединяющей концы отрезка $(a, \varphi(a))$ и $(b, \varphi(b))$, с осью Ox :

$$x_{n+1} = a_n - \varphi(a_n) \frac{b_n - a_n}{\varphi(b_n) - \varphi(a_n)}.$$

Найденную точку x_{n+1} берут вместо того конца отрезка, на котором знак функции совпадает со знаком $\varphi(x_{n+1})$ (так, чтобы корень оставался внутри новой пары концов). В качестве критерия остановки используется $|x_{n+1} - x_n| < \varepsilon_1$.

Метод хорд имеет, как правило, более высокую скорость сходимости, чем метод деления пополам, поскольку использует не только знаки функции на концах, но и их величины. На тесте $x^2 - 2 = 0$ при $\varepsilon = 10^{-8}$ метод хорд потребовал 11 итераций против 27 у деления пополам (см. раздел 6).

Обоснование выбора ε_1

Для метода деления пополам на отрезке $[a, b]$ погрешность после n итераций равна [1]:

$$|x_n - x^*| \leq \frac{b - a}{2^n}.$$

Чтобы гарантировать $\varepsilon_1 = 10^{-5}$ на самом широком из трёх отрезков (длина $[1, 3] = 2$), нужно:

$$n \geq \log_2 \frac{2}{10^{-5}} = \log_2(200000) \approx 17,6 \Rightarrow n \geq 18.$$

Для метода хорд число итераций при той же ε_1 меньше за счёт более высокого порядка сходимости. Такое число итераций в обоих случаях выполняется менее чем за миллисекунду. Поскольку $\varepsilon_1 = 10^{-5} \ll \varepsilon = 10^{-3}$, погрешность нахождения корней практически не влияет на точность площади.

Формула для площади

На участке $[x_A, x_B]$ порядок функций следующий: $f_2 < f_3 < f_1$ (верхняя граница — f_1 , нижняя — f_3). На участке $[x_B, x_C]$: $f_3 < f_2 < f_1$ (нижняя граница — f_2). Таким образом:

$$S = \int_{x_A}^{x_B} (f_1(x) - f_3(x)) dx + \int_{x_B}^{x_C} (f_1(x) - f_2(x)) dx.$$

Обоснование выбора ε_2

Погрешность метода средних прямоугольников оценивается как [1]:

$$|I - I_n| \leq \frac{M_2(b - a)^3}{24n^2},$$

где $M_2 = \max_{[a,b]} |f''(x)|$. В программе реализован критерий Рунге:

$$\frac{|I_{2n} - I_n|}{3} < \varepsilon_2.$$

При $\varepsilon_2 = 10^{-5}$ фактическая погрешность каждого интеграла не превышает 10^{-5} , что в сумме по двум слагаемым даёт не более $2 \cdot 10^{-5} \ll 10^{-3}$.

Графики функций

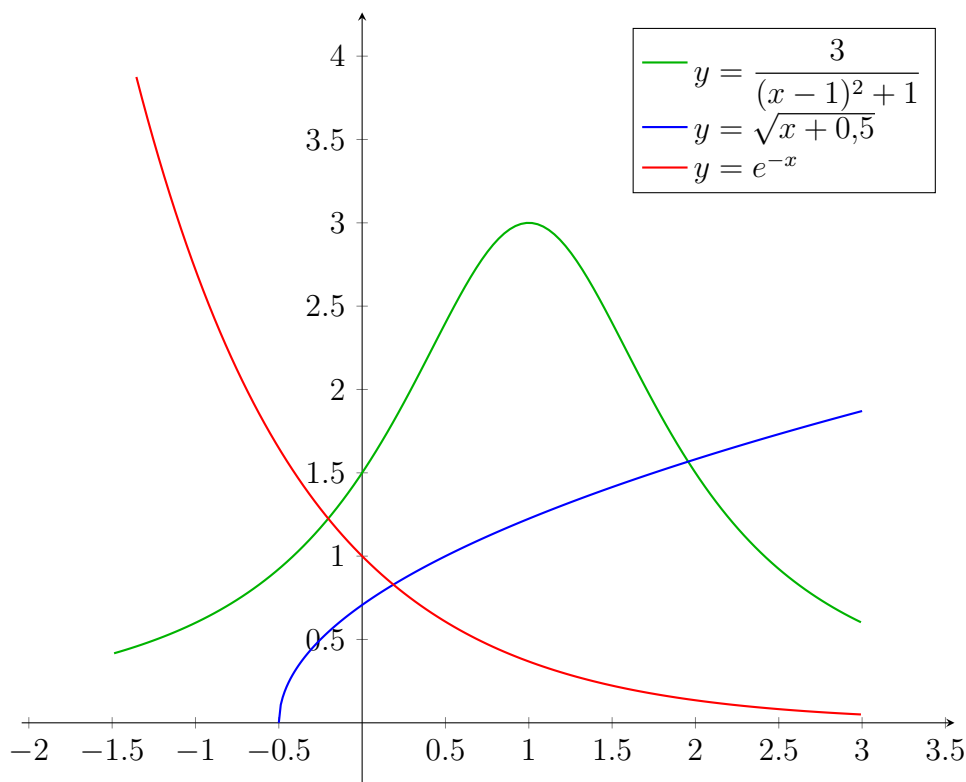


Рис. 1: Графики функций f_1 , f_2 , f_3

Результаты экспериментов

Программа запускалась с $\varepsilon_1 = \varepsilon_2 = 10^{-5}$.

Пересечение	x	y
f_1 и f_3 (точка А)	-0,2033	1,2255
f_2 и f_3 (точка В)	0,1874	0,8291
f_1 и f_2 (точка С)	1,9562	1,5672

Таблица 1: Координаты вершин фигуры

Площадь фигуры:

$$S_{AB} = \int_{-0,2033}^{0,1874} (f_1 - f_3) dx \approx 0,18877, \quad S_{BC} = \int_{0,1874}^{1,9562} (f_1 - f_2) dx \approx 2,14982,$$

$$S = S_{AB} + S_{BC} \approx \mathbf{2,3386}.$$

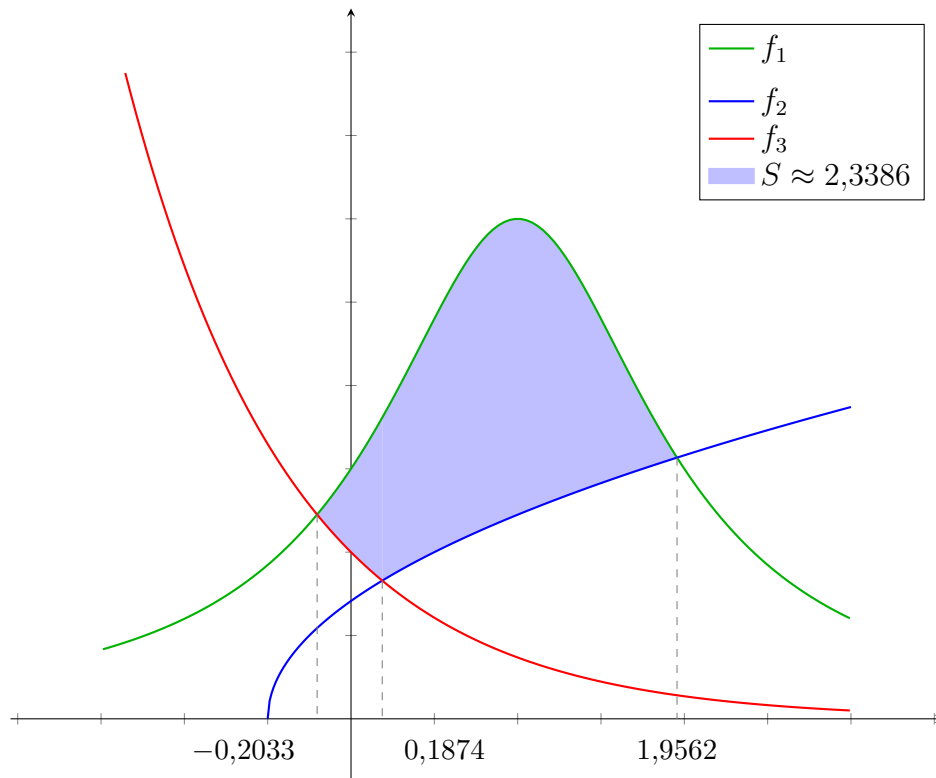


Рис. 2: Закрашенная фигура

Структура программы и спецификация функций

Программа состоит из шести модулей.

- **f1.asm** — вычисление $f_1(x) = 3/((x-1)^2 + 1)$ средствами x87 FPU.
- **f2.asm** — вычисление $f_2(x) = \sqrt{x+0.5}$ инструкцией `fsqrt`.
- **f3.asm** — вычисление $f_3(x) = e^{-x}$ через тождество $e^y = 2^{y \log_2 e}$ с помощью `fldl2e`, `f2xm1`, `fscale`.
- **root.c** — функция `root(f, g, a, b, eps)` для уравнения $f(x) = g(x)$. Содержит две реализации: метод деления отрезка пополам (выбирается по умолчанию) и метод хорд (выбирается через макрос `METHOD_CHORDS`, переключение — директивами `#ifdef`). Хранит число итераций в глобальной переменной `g_iterations`.
- **integral.c** — функция `integral(f, a, b, eps)`: метод средних прямоугольников с остановкой по критерию Рунге.
- **main.c** — разбор аргументов командной строки, вычисление площади, режимы тестирования.

Все функции f_1 , f_2 , f_3 и их вызывающие стороны следуют соглашению **cdecl**: аргумент `double` передаётся на стеке по адресу `[ebp+8]`, результат возвращается в регистре `st0` сопроцессора x87.

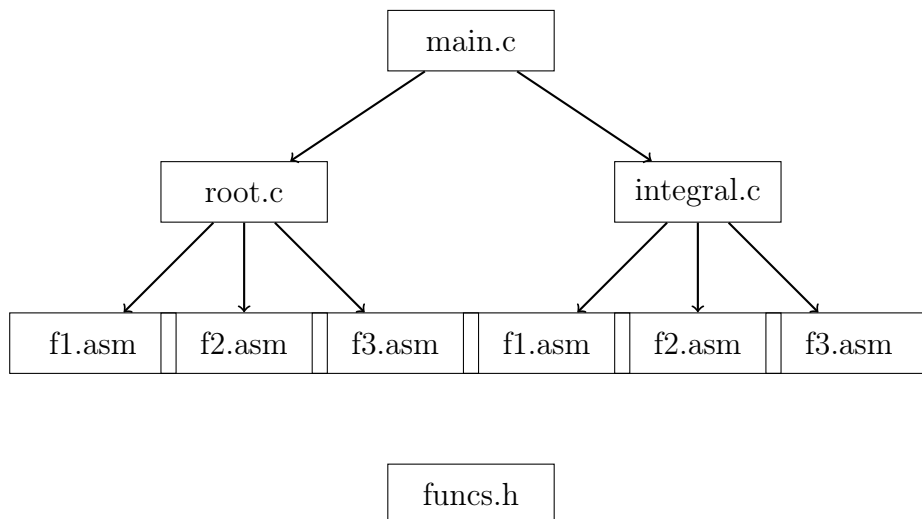


Рис. 3: Зависимости между модулями

Зависимости между модулями показаны на рис. 3.

Сборка программы (Make-файл)

Сборка производится командой `make`. Ниже приведён Makefile:

```
CC      = gcc
NASM    = nasm
CFLAGS  = -m32 -std=c99
NFLAGS  = -f elf32
OBJECTS = main.o root.o integral.o f1.o f2.o f3.o

ifeq ($(METHOD),CHORDS)
    CFLAGS += -DMETHOD_CHORDS
endif

all: prog

prog: $(OBJECTS)
    $(CC) $(CFLAGS) -o prog $(OBJECTS) -lm

main.o:      src/main.c include/funcs.h
    $(CC) $(CFLAGS) -c -o main.o src/main.c

root.o:      src/root.c include/funcs.h
    $(CC) $(CFLAGS) -c -o root.o src/root.c

integral.o:  src/integral.c include/funcs.h
    $(CC) $(CFLAGS) -c -o integral.o src/integral.c

f1.o:  asm/f1.asm
    $(NASM) $(NFLAGS) -o f1.o asm/f1.asm

f2.o:  asm/f2.asm
    $(NASM) $(NFLAGS) -o f2.o asm/f2.asm

f3.o:  asm/f3.asm
    $(NASM) $(NFLAGS) -o f3.o asm/f3.asm

clean:
    rm -f *.o prog
```

Объектные файлы `f1.o`, `f2.o`, `f3.o` собираются NASM в формате `elf32`. Объектные файлы Си-модулей собираются GCC с флагом `-m32` для 32-разрядного кода. Финальная сборка выполняется gcc с подключением `libm` для функций стандартной математической библиотеки, используемых в `integral.c`. Выбор метода поиска корня осуществляется переменной `METHOD`: по умолчанию собирается версия с делением пополам, команда `make METHOD=CHORDS` добавляет к `CFLAGS` флаг `-DMETHOD_CHORDS`, и компилируется ветка с методом хорд.

Отладка программы, тестирование функций

Тестирование root

Функция `root` тестируется командой `./prog -test-root`.

Тест 1. Нахождение $\sqrt{2}$ как корня $x^2 - 2 = 0$ на $[1, 2]$. Аналитический ответ: $x^* = \sqrt{2} \approx 1,41421356$. Функция $\varphi(x) = x^2 - 2$: $\varphi(1) = -1 < 0$, $\varphi(2) = 2 > 0$ — знак меняется, оба метода применимы. Для деления пополам теоретическое число итераций:

$$n \geq \log_2 \frac{2-1}{10^{-8}} \approx 27 \text{ итераций.}$$

Результаты программы при $\varepsilon = 10^{-8}$:

- деление пополам (`make`): $x \approx 1,4142135642$, итераций 27;
- метод хорд (`make METHOD=CHORDS`): $x \approx 1,4142135605$, итераций 11.

Оба значения имеют ошибку $< 10^{-8}$. Метод хорд сошёлся примерно в 2,5 раза быстрее за счёт более высокого порядка сходимости.

Тест 2. Нахождение корня $\cos(x) = 0$ на $[1, 2]$ через `root(cos_f, zero, 1, 2, 1e-7)`. Аналитический ответ: $x^* = \pi/2 \approx 1,5707963$. Знак: $\cos(1) \approx 0,54 > 0$, $\cos(2) \approx -0,41 < 0$ — применим.

Тест 3. Нахождение корня $e^x - 2 = 0$ на $[0, 1]$. Аналитический ответ: $x^* = \ln 2 \approx 0,693147$. Знак: $e^0 - 2 = -1 < 0$, $e^1 - 2 \approx 0,718 > 0$ — применим.

Тестирование integral

Функция `integral` тестируется командой `./prog -test-integral`.

Тест 1. $\int_0^1 x dx = 0,5$ (прямая, тривиальный). Результат: 0,5000000000.

Тест 2. $\int_0^1 x^2 dx = \frac{1}{3} \approx 0,333333$. Функция x^2 — квадратичная, метод средних прямоугольников интегрирует её точно за одно разбиение (нет погрешности за счёт симметрии). Результат: 0,3333333333.

Тест 3. $\int_0^\pi \sin(x) dx = 2$. Функция нелинейная, интеграл вычисляется с несколькими разбиениями. Результат: 2,0000000000 (до 10 знаков).

Программа на Си и на Ассемблере

Исходные тексты программы приложены к отчёту в архиве. Архив содержит:

- `src/main.c`, `src/root.c`, `src/integral.c` — код на Си;
- `asm/f1.asm`, `asm/f2.asm`, `asm/f3.asm` — ассемблерные реализации функций варианта;
- `include/funcs.h` — заголовочный файл с объявлениями всех функций;
- `Makefile` — сценарий сборки.

Анализ допущенных ошибок

В ходе разработки были допущены следующие ошибки.

1. Неверные границы поиска корней. Изначально отрезок для $f_1 \cap f_2$ был взят как $[-2, 0]$, не учитывая, что $f_2(x) = \sqrt{x + 0,5}$ не определена при $x < -0,5$. Это приводило к вычислению $\sqrt{\text{отрицательного числа}}$ и возврату NaN. Ошибка устранена аналитической проверкой области определения функций до выбора отрезков.

2. Стек x87 после fscale. В реализации f_3 инструкция `fscale` оставляет на стеке два значения: результат и исходное n . Без явного удаления n (инструкция `fstp st1`) стек FPU переполнялся после нескольких вызовов функции, и результаты становились некорректными. После добавления `fstp st1` ошибка исчезла.

3. Кеш Fa в методе деления пополам. В первоначальной реализации значение $F(a) = f(a) - g(a)$ вычислялось заново на каждой итерации, хотя правый конец отрезка обновляется только при $F_a \cdot F_{mid} < 0$, а иначе меняется только a . После кеширования F_a число вызовов функций сократилось примерно вдвое.

Список литературы

- [1] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 — Москва: Наука, 1985.